

Steam Library Manager

JWT Authentication & RBAC Implementation

Modul M223

Philipp Seewer

Projektübersicht

Vollständige Web-Anwendung mit JWT-Authentifizierung, RBAC und sicherer Datenbankintegration. Entwickelt mit modernen Best Practices für Backend (Spring Boot 3), Frontend (React 18) und Datenbanken (MySQL 8).

Inhaltsverzeichnis

- 1. Projektübersicht
- 2. Anforderungen & User Stories
- 3. Arbeitsplan & Aufwandschätzung
- 4. Systemarchitektur
- 5. JWT Authentication & Sicherheitskonzept
- 6. Implementierungsdetails
- 7. Testing & Qualitätssicherung
- 8. Datenbank & Transaktionen
- 9. Betrieb & Installation
- 10. Arbeitsjournal
- 11. Auswertung & Reflexion

1. Projektübersicht

Die Steam Library Manager-Anwendung ist ein vollständiges Web-Anwendungssystem mit modernen Sicherheitsstandards:

- React 18 Frontend mit responsivem Design
- Spring Boot 3 REST API Backend
- JWT-basierte Authentifizierung und RBAC
- MySQL 8 Datenbank mit Docker
- Umfassende Testabdeckung

Projektzielsetzung

Das Projekt demonstriert eine sichere, skalierbare Webanwendung. Benutzer können sich registrieren, anmelden und ihre Spiele-Bibliothek verwalten mit strikten Zugriffsrollen.

Technologie-Stack

Komponente	Technologie
Backend	Java 11, Spring Boot 3, Spring Security, JWT
Frontend	React 18, Vite, Context API, React Router v7
Datenbank	MySQL 8 in Docker
Build	Maven (Backend), npm (Frontend)

Projektbasierung & Zusammenhang

Dieses Projekt baut auf Modul M295 (Datenbank-Design und SQL) auf. Das ursprüngliche Datenmodell stammt aus M295, wurde aber für JWT-Authentifizierung und RBAC angepasst.

2. Anforderungen & User Stories

Funktionale Anforderungen

- Spiele persistent in MySQL speichern
- CRUD-Operationen über REST API
- Benutzerregistrierung und Anmeldung mit JWT
- ROLE_USER (Lesezugriff), ROLE_ADMIN (voller Zugriff)
- Sichere Frontend-Routes mit PrivateRoute

User Stories

User Story 1: Spiel erfassen (ROLE_ADMIN)

Als Administrator möchte ich ein neues Spiel erfassen, damit ich die Bibliothek erweitern kann.

Akzeptanzkriterien:

- POST /api/games erstellt Spiel (HTTP 201)
- Nur ROLE_ADMIN darf POST ausführen

3. Arbeitsplan & Aufwandschätzung

ID	Beschreibung	Dauer	Status
AP1	Projektplanung & Setup	3h	✓
AP2	Backend: Spring Boot & JPA	5h	✓
AP3	JWT Authentication	8h	✓
AP4	RBAC & Spring Security	6h	✓
AP5	Frontend React	7h	✓
AP6	Testing & Bugs	6h	✓
AP7	Dokumentation	6h	✓
Aktivität		Aufwand	Prozent
Implementierung		24h	60%
Testing		8h	20%
Dokumentation		8h	20%

Gesamtaufwand: ~40 Stunden

4. Systemarchitektur

Backend-Architektur

Mehrstufige Architektur mit Separation of Concerns:

- Controller Layer: GameController, AuthController
- Service Layer: GameService - Business Logic
- Repository Layer: GameRepository, UserRepository
- Security Layer: JwtAuthFilter, SecurityConfig

Frontend-Architektur

Single Page Application (SPA) mit React 18:

- AuthContext: Globaler State
- PrivateRoute: Geschützte Seiten
- Login/Register: Authentifizierungs-Formulare
- Games: CRUD-Verwaltung

5. JWT Authentication & Sicherheitskonzept

JWT-Token Struktur

JWT bestehen aus drei Base64-kodierten Teilen:

- Header: alg: HS256, typ: JWT
- Payload: sub (username), iat, exp, roles
- Signature: HMAC-SHA256(Header + Payload, SECRET_KEY)

Sicherheitsmassnahmen

- Passwort-Hashing mit BCrypt
- JWT-Token mit 24h Ablaufzeit
- HMAC-SHA256 Signatur-Verifizierung
- CORS für trusted origins
- Stateless Session-Handling
- Bean Validation für Input

6. Implementierungsdetails

Datenmodell (Game Entity)

Feld	Typ	Constraints	Beschreibung
id	BIGINT	PK, AUTO	Primärschlüssel
title	VARCHAR(255)	NOT NULL	Spieltitel
steamAppId	INT	UNIQUE	Steam App ID
user_id	BIGINT	FK, NOT NULL	Fremdschlüssel

API-Endpunkte

Authentifizierungs-Endpunkte (öffentlich):

- POST /api/auth/signup - Registrieren
- POST /api/auth/signin - JWT erhalten

Game-Endpunkte (geschützt):

- GET /api/games - Abrufen
- POST /api/games - Erstellen (ADMIN)
- PUT /api/games/{id} - Aktualisieren (ADMIN)
- DELETE /api/games/{id} - Löschen (ADMIN)

7. Testing & Qualitätssicherung

ID	Testmethode	Erwartung
UT-01	whenAddingValidGame_thenGamelsSaved	Game gespeichert
UT-02	whenGettingAllGames_thenReturnCorrectCount	3 Spiele
IT-01	getAll_returnsOkAndArray	HTTP 200
IT-02	create_validGame_returnsCreatedAndId	HTTP 201

Insgesamt: 13 Tests (8 Service + 5 Controller) - Alle bestanden ✓

Testfall	Aktion	Ergebnis
Registrierung	Neue Benutzer	✓ HTTP 200
Login	Gültige Credentials	✓ JWT erhalten
GET /api/games	Mit JWT-Token	✓ HTTP 200
POST (USER)	USER-Rolle	✓ HTTP 403
PrivateRoute	Ohne Login	✓ Weiterleitung

8. Datenbank & Transaktionen

Datenbank-Setup

```
docker run --name steamlibrary-db -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=steamlibrary -p 3306:3306 -d mysql:8
```

Transaktionen & Konsistenz

Kritische Operationen mit @Transactional:

- GameService.createGame() - mit Validierung
- GameService.updateGame() - mit Rollback
- GameService.deleteGame() - mit Konsistenzprüfung
- AuthController.register() - mit Duplikatsprüfung

Constraints:

- Unique Constraints für username, email, steamAppId
- Foreign Key Constraints für user_id
- NOT NULL Constraints für kritische Felder
- Bean Validation: @NotBlank, @NotNull, @PositiveOrZero

9. Betrieb & Installation

Voraussetzungen

- Java 11+
- Node.js 16+
- Docker & Docker Compose
- MySQL 8

Starten

Backend: `cd Backend && ./mvnw spring-boot:run` (<http://localhost:8080>)

Frontend: `cd Front-End && npm install && npm run dev` (<http://localhost:5173>)

Benutzername	Passwort	Rolle
admin_user	password123	ROLE_ADMIN
user1	password123	ROLE_USER

10. Arbeitsjournal

Datum	Aktivität	Std.	Notizen
19.02	Setup & Planung	3	Backend/Frontend
20.02-24.02	JWT-Impl.	8	JwtUtil, Filter, Config
25.02-28.02	RBAC & Test	8	Unit & Integration
01.03-14.03	Frontend	10	AuthContext, PrivateRoute
15.03-20.03	Dokumentation	8	Dokumentation & Präsentation

11. Auswertung & Reflexion

Anforderung	Soll	Ist	Status
JWT Auth	24h Token	24h Token	✓
RBAC	2+ Rollen	2 Rollen	✓
Backend Tests	min. 2	13 Tests	✓✓
Frontend Tests	min. 2	3+ manuell	✓
Dokumentation	Umfassend	11 Sections	✓✓
Git Commits	Aussagekräftig	2 detailliert	✓

Erreichte Ziele

- ✓ JWT-Authentifizierung mit stateless Sessions
- ✓ Rollenbasierte Zugriffskontrolle (RBAC)
- ✓ Passwort-Hashing mit BCrypt
- ✓ 13 automatisierte Tests (alle bestanden)
- ✓ Responsive React Frontend mit Context API
- ✓ Umfassende Dokumentation und Testabdeckung

Herausforderungen & Lösungen

JWT-Token Rollen-Claims

- Problem: Rollen nicht in Token → 403 Fehler
- Lösung: JwtUtil.generateToken() mit 'roles'; JwtAuthFilter extrakt Rollen

API-Endpoint Mismatch

- Problem: /games vs /api/games → Konflikt
- Lösung: Standardisierung auf /api/games

user_id NULL Constraints

- Problem: user_id nicht gesetzt → DB Fehler
- Lösung: GameController erweitert; auto-population

Verbesserungspotential

- Token Refresh Mechanism
- Rate Limiting auf Auth-Endpunkten
- Audit Logging für sensitive Ops
- WebSocket Support für Real-time
- E2E Tests mit Cypress/Playwright

Projektreflexion

Nach dem Entwicklungsprozess zeigten sich Erkenntnisse für Optimierungen:

- JWT-Komplexität höher als erwartet
- Externe Dependencies hätten vermieden werden können
- Scope-Creep durch zu ambitioniertes Design
- Alternative Projekte hätten bessere didaktische Struktur geboten

Trotzdem wurde das Projekt erfolgreich umgesetzt und bietet wertvollen Input für zukünftige Projekte.